TECHNICAL DELIVERABLE

Project Architecture Report

Bloom Parfums — Full-Stack E-Commerce Platform

Next.js 14 App Router | Laravel REST API | MySQL | JWT / Sanctum | Redis | AWS S3

Prepared by: AI Lead Architect **Date:** February 24, 2026 **Version:** 1.0 **Status:** Final

Project Architecture Report — Bloom Parfums

Prepared by: AI Lead Architect
Date: February 24, 2026
Stack: Next.js 14 (App Router) · Laravel (REST API) · MySQL
Delivery model: SPA + SSR frontend, headless API backend, JWT authentication

Table of Contents

1. Frontend Analysis

1.1 Route Map

Route	Rendering	Auth	UI Pattern	Purpose
`/`	SSG	Guest	Landing / Marketing	Homepage with hero, sections, bestsellers
`/login`	CSR	Guest only	Auth form	JWT login, redirects to `/dashboard`
`/register`	CSR	Guest only	Auth form	Account creation with field-level validation
`/dashboard`	SSR	Required	Profile / Stats	User account hub with order/wishlist counters
`/collection`	CSR	Guest	Product list + Filters	Browse catalog with sidebar filtering
`/product/[id]`	Dynamic	Guest	Product detail	Single product view (route exists, page not yet built)
`/wishlist`	CSR	Implied	Product grid	Saved products, sort, add to cart
`/checkout`	CSR	Required	Wizard step 1/4	Shipping address + delivery method selection
`/success`	CSR	Required	Confirmation	Post-order label & confirmation step 2/4
`/track-order`	CSR	Guest	Form	Order tracking by number step 3/4
`/order-status`	CSR	Guest	Timeline	Delivery timeline + shipment contents step 4/4
`/feedback`	CSR	Required	Review form	Post-delivery rating per product

1.2 Component Inventory

Component	Type	Responsibility
`Header`	Layout	Global nav, search, cart icon, filter trigger, login/wishlist links
`Footer`	Layout	Site links, legal, social
`CartDrawer`	Modal/Overlay	Slide-in cart with qty controls, coupon, totals, checkout CTA
`FilterModal`	Modal/Drawer	Slide-in filter panel: brand, price range, gender
`ReviewModal`	Modal	Product review form: star rating, text, photo upload
`DashboardClient`	Page Client	Renders SSR-prefetched user data, logout action
`WishlistOverlay`	Overlay	Quick wishlist preview
`HeroSection`	Page Section	Full-width hero banner
`BrandLogos`	Page Section	Brand carousel/grid
`BestSellers`	Page Section	Product cards row
`CategoriesSection`	Page Section	Category tiles
`UniversSection`	Page Section	Thematic/lifestyle editorial
`ValentinesSection`	Page Section	Promotional seasonal banner
`CustomerReviewsSection`	Page Section	Social proof testimonials
`LoadingSpinner`	Utility	Shared loading indicator
`SectionContainer`	Utility	Wrapper with consistent max-width + padding
`Navbar`	Layout (Dashboard)	Admin/user-specific top navigation

1.3 Services & Auth Layer

The service layer (`services/api.ts`) is well-structured:

- **Axios instance** with `baseURL`, `withCredentials`, 15s timeout.
- **Request interceptor** attaches `Bearer` token from cookie to every outgoing request.
- **Response interceptor** handles 401 → auto-refresh → retry once → redirect to `/login` on failure.
- `lib/auth.ts` provides `setAuthToken`, `clearAuthToken`, `isAuthenticated`, and `serverFetch` for Server Components.

1.4 State Management Assessment

Concern	Current approach	Assessment
Auth token	`js-cookie` + cookie read in server component	Adequate for small scale
Cart state	Local `useState` in `CartDrawer` (hardcoded)	Not connected to backend — critical gap
Wishlist state	Local component state (hardcoded)	Not connected to backend — critical gap
Filters	Local `useState` in `FilterModal` / Collection page	Not connected — will need URL-based state
User session	SSR prefetch via `serverFetch` in `DashboardPage`	Correct pattern

2. UI/UX Audit

2.1 Strengths

- **Consistent brand language.** The serif/italic typography, gold `#cda873` accent, and warm off-white backgrounds are applied coherently across all pages.
- **Component reuse.** Header/Footer wrap every public page. Section components are isolated and composable.
- **Post-purchase flow.** The 5-step flow (Checkout → Success → Track → Status → Feedback) is architecturally sound and provides a clear user journey.
- **Progressive disclosure.** Modals (Cart, Filter, Review) avoid full-page navigation for secondary actions.

2.2 Critical UX Gaps

Issue	Page(s)	Severity	Recommendation
All cart/wishlist data is hardcoded	Cart, Wishlist, Header count	**Critical**	Wire to backend API; use React Context or Zustand for cart state
No loading states	All CSR pages	**High**	Add skeleton loaders and `<Suspense>` boundaries
No empty states	Wishlist, Cart, Order history	**High**	Design and implement empty state components
No error boundaries	All pages	**High**	Add `error.tsx` in App Router at segment level
Filter state not persisted in URL	Collection, FilterModal	**High**	Use `useSearchParams` with URL query strings for shareable filtered URLs
Checkout form has no validation	`/checkout`	**High**	Add real-time validation before submit
Product cards have no "Add to cart" feedback	Collection, Wishlist	**Medium**	Add optimistic UI update + toast notification
`/product/[id]` route has no page	product detail	**High**	Critical commerce page missing
Dashboard stats show `–`	`/dashboard`	**Medium**	Fetch real counts from API
No pagination on Collection/Wishlist	Collection, Wishlist	**Medium**	Implement infinite scroll or page numbers
Coupon system is UI-only	CartDrawer	**Medium**	Wire to backend coupon validation endpoint
Success page has no real order data	`/success`	**High**	Populate from order creation API response
Order status timeline is static	`/order-status`	**High**	Poll or subscribe to order status from backend
No mobile navigation menu	Header	**Medium**	Add hamburger menu for small viewports
Login/Register pages skip Header/Footer	`/login`, `/register`	**Low**	Intentional, but consider adding brand logo link

2.3 Accessibility Issues

- Interactive `<div>` elements used for product cards (clickable without `role="button"` or keyboard handlers).
- Filter checkboxes use custom radio UI without proper ARIA roles.
- Color contrast on gold-on-white text (`#cda873` on `#fdfbf5`) may fail WCAG AA at small font sizes.
- Images in many sections lack meaningful `alt` text.
- Modal focus trap is not implemented in `CartDrawer` , `FilterModal` , or `ReviewModal` .

2.4 Performance Risks

- `Math.random()` used inside JSX render for histogram bars in `CollectionPage` — causes hydration mismatch and unnecessary re-renders on every render cycle.
- `Array(8).fill( ... )` in `WishlistPage` is hardcoded; will be replaced by API data but currently has no memoization.
- `Image` components using Unsplash CDN URLs should add `sizes` props to prevent over-fetching on mobile.

---

3. Functional Requirements

3.1 Core Features (P0)

- User registration and login (JWT-based)
- Product catalog browsing with filtering (brand, price, gender/category)
- Product detail page (quantity, size selection, add to cart)
- Wishlist (add/remove, persistent across sessions)
- Shopping cart (add/remove/update qty, coupon code)
- Checkout (shipping address, delivery method selection)
- Order creation and confirmation
- Order tracking (by order number, guest + auth)
- Order status timeline (real-time or polled)
- Post-purchase product review + photo upload

3.2 Secondary Features (P1)

- User dashboard with order history
- Loyalty points system (points counter in dashboard)
- Coupon/promo code validation
- Brand/category landing pages
- Seasonal promotion sections (Valentines, etc.)

3.3 Admin Features (P2)

- Product management (CRUD) — implied by `admin` role in `User` type
- Order management and status updates
- Brand and category management
- Coupon management
- Customer management
- Review moderation

3.4 Role Matrix

Action	Guest	Customer	Admin
Browse catalog	✅	✅	✅
View product detail	✅	✅	✅
Add to cart	✅	✅	✅
Place order	❌	✅	✅
View own orders	❌	✅	✅
Submit review	❌	✅ (post-purchase)	✅
Wishlist	❌	✅	✅
Manage products	❌	❌	✅
Manage orders	❌	❌	✅

4. Data Models

4.1 User

Purpose: Authenticated store customer or admin operator.

Field	Type	Required	Notes
id	bigint	✅	PK, auto-increment
name	string(100)	✅	Full name
email	string(191)	✅	Unique, indexed
password	string	✅	Hashed (bcrypt)
role	enum('customer','admin')	✅	Default: customer
email_verified_at	timestamp	❌	Nullable
loyalty_points	int	✅	Default: 0
avatar_url	string	❌	Profile picture
phone	string(30)	❌	
created_at / updated_at	timestamp	✅	
deleted_at	timestamp	❌	Soft delete

Relationships: has many `Order`, `Address`, `Review`, `WishlistItem`

4.2 Product

Purpose: Sold fragrance or body care item.

Field	Type	Required	Notes
id	bigint	✅	PK
name	string(200)	✅	Display name
slug	string(200)	✅	Unique URL key
description	text	❌	Rich product description
price	decimal(10,2)	✅	Current sale price
original_price	decimal(10,2)	❌	For discount display
stock	int	✅	Default: 0
brand_id	bigint	✅	FK → brands
category_id	bigint	✅	FK → categories
gender	enum('men','women','unisex','kids')	✅	For gender filter
image_url	string	❌	Primary image
is_featured	boolean	✅	BestSellers section
is_active	boolean	✅	Published flag
created_at / updated_at	timestamp	✅	
deleted_at	timestamp	❌	Soft delete

Relationships: belongs to `Brand`, `Category`; has many `ProductImage`, `OrderItem`, `WishlistItem`, `Review`

4.3 Brand

Purpose: Fragrance brand shown in BrandLogos section and product filter.

Field	Type	Required
id	bigint	✅
name	string(100)	✅
slug	string(100)	✅
logo_url	string	❌
created_at / updated_at	timestamp	✅

4.4 Category

Purpose: Product classification (e.g., Body Butter, Mist, Cream).

Field	Type	Required
id	bigint	✅
name	string(100)	✅
slug	string(100)	✅
parent_id	bigint	❌
image_url	string	❌
created_at / updated_at	timestamp	✅

4.5 Address

Purpose: Saved shipping addresses for a user.

Field	Type	Required	Notes
id	bigint	✅	
user_id	bigint	✅	FK → users
first_name	string(100)	✅	
last_name	string(100)	✅	
phone	string(30)	✅	
address_line	string(300)	✅	
city	string(100)	✅	
quartier	string(100)	❌	Local area — Morocco-specific
zip_code	string(20)	❌	
country_code	char(3)	✅	Default: MAR
is_default	boolean	✅	Default: false
created_at / updated_at	timestamp	✅	

4.6 Order

Purpose: A customer's purchase transaction.

Field	Type	Required	Notes
id	bigint	✅	
order_number	string(20)	✅	Unique, e.g. `LX-8921-Q`
user_id	bigint	❌	Nullable for guest checkout
address_id	bigint	✅	FK → addresses (snapshot at order time)
shipping_method_id	bigint	✅	FK → shipping_methods
coupon_id	bigint	❌	FK → coupons
status	enum	✅	pending/confirmed/dispatched/shipped/delivered/cancelled
subtotal	decimal(10,2)	✅	
shipping_cost	decimal(10,2)	✅	
discount	decimal(10,2)	✅	Default: 0
total	decimal(10,2)	✅	
notes	text	❌	
created_at / updated_at	timestamp	✅	
deleted_at	timestamp	❌	

4.7 OrderItem

Purpose: Line-item inside an order.

Field	Type	Required
id	bigint	✅
order_id	bigint	✅
product_id	bigint	✅
product_name	string	✅
unit_price	decimal(10,2)	✅
quantity	int	✅
created_at / updated_at	timestamp	✅

4.8 OrderStatusHistory

Purpose: Append-only log of status changes — drives the timeline UI.

Field	Type	Required
id	bigint	✅
order_id	bigint	✅
status	enum (same as Order.status)	✅
note	string	❌
created_at	timestamp	✅

4.9 WishlistItem

Purpose: Persisted wishlist for authenticated users.

Field	Type	Required
id	bigint	✅
user_id	bigint	✅
product_id	bigint	✅
created_at	timestamp	✅

Unique composite index on `(user_id, product_id)`.

4.10 Review

Purpose: Post-purchase product rating and comment.

Field	Type	Required
id	bigint	✅
user_id	bigint	✅
product_id	bigint	✅
order_item_id	bigint	✅
rating	tinyint (1-5)	✅
body	text	❌
is_approved	boolean	✅
created_at / updated_at	timestamp	✅

4.11 ShippingMethod

Purpose: Delivery options with pricing (Free, Région, National).

Field	Type	Required
id	bigint	✅
name	string(100)	✅
description	string	❌
price	decimal(10,2)	✅
zone	string(100)	❌
is_active	boolean	✅
created_at / updated_at	timestamp	✅

4.12 Coupon

Purpose: Discount codes applied at cart.

Field	Type	Required	Notes
id	bigint	✅	
code	string(50)	✅	Unique, uppercase
type	enum('fixed','percent')	✅	
value	decimal(10,2)	✅	
min_order	decimal(10,2)	❌	Minimum cart total
usage_limit	int	❌	Null = unlimited
used_count	int	✅	Default: 0
expires_at	timestamp	❌	
is_active	boolean	✅	
created_at / updated_at	timestamp	✅	

4.13 ProductImage

Purpose: Gallery images per product.

Field	Type	Required
id	bigint	✅
product_id	bigint	✅
url	string	✅
sort_order	int	✅
created_at / updated_at	timestamp	✅

5. Database Schema

```
-- ── brands ──────────────────────────────────────────
CREATE TABLE brands (
  id         BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  name       VARCHAR(100)  NOT NULL,
  slug       VARCHAR(100)  NOT NULL UNIQUE,
  logo_url   VARCHAR(500),
  created_at TIMESTAMP     NULL DEFAULT NULL,
  updated_at TIMESTAMP     NULL DEFAULT NULL
);

-- ── categories ──────────────────────────────────────
CREATE TABLE categories (
  id         BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  parent_id  BIGINT UNSIGNED NULL,
  name       VARCHAR(100)  NOT NULL,
  slug       VARCHAR(100)  NOT NULL UNIQUE,
  image_url  VARCHAR(500),
  created_at TIMESTAMP     NULL DEFAULT NULL,
  updated_at TIMESTAMP     NULL DEFAULT NULL,
  FOREIGN KEY (parent_id) REFERENCES categories(id) ON DELETE SET NULL,
  INDEX idx_categories_parent (parent_id)
);

-- ── users ───────────────────────────────────────────
CREATE TABLE users (
  id                 BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  name               VARCHAR(100)  NOT NULL,
  email              VARCHAR(191)  NOT NULL UNIQUE,
  email_verified_at  TIMESTAMP    NULL,
  password           VARCHAR(255)  NOT NULL,
  role               ENUM('customer','admin') NOT NULL DEFAULT 'customer',
  loyalty_points     INT UNSIGNED  NOT NULL DEFAULT 0,
  avatar_url         VARCHAR(500),
  phone              VARCHAR(30),
  remember_token     VARCHAR(100),
  created_at         TIMESTAMP    NULL DEFAULT NULL,
  updated_at         TIMESTAMP    NULL DEFAULT NULL,
  deleted_at         TIMESTAMP    NULL DEFAULT NULL,
  INDEX idx_users_email (email),
  INDEX idx_users_role  (role)
);

-- ── products ────────────────────────────────────────
CREATE TABLE products (
  id             BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  brand_id       BIGINT UNSIGNED NOT NULL,
  category_id    BIGINT UNSIGNED NOT NULL,
  name           VARCHAR(200)  NOT NULL,
  slug           VARCHAR(200)  NOT NULL UNIQUE,
  description    TEXT,
  price          DECIMAL(10,2) NOT NULL,
```

```
  original_price  DECIMAL(10,2),
  stock           INT UNSIGNED  NOT NULL DEFAULT 0,
  gender          ENUM('men','women','unisex','kids') NOT NULL DEFAULT 'unisex',
  image_url       VARCHAR(500),
  is_featured     BOOLEAN       NOT NULL DEFAULT FALSE,
  is_active       BOOLEAN       NOT NULL DEFAULT TRUE,
  created_at      TIMESTAMP     NULL DEFAULT NULL,
  updated_at      TIMESTAMP     NULL DEFAULT NULL,
  deleted_at      TIMESTAMP     NULL DEFAULT NULL,
  FOREIGN KEY (brand_id)    REFERENCES brands(id)     ON DELETE RESTRICT,
  FOREIGN KEY (category_id) REFERENCES categories(id) ON DELETE RESTRICT,
  INDEX idx_products_brand    (brand_id),
  INDEX idx_products_category (category_id),
  INDEX idx_products_gender   (gender),
  INDEX idx_products_featured (is_featured),
  INDEX idx_products_price    (price)
);

-- ── product_images ──────────────────────────────────
CREATE TABLE product_images (
  id          BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  product_id  BIGINT UNSIGNED NOT NULL,
  url         VARCHAR(500)  NOT NULL,
  sort_order  TINYINT UNSIGNED NOT NULL DEFAULT 0,
  created_at  TIMESTAMP     NULL DEFAULT NULL,
  updated_at  TIMESTAMP     NULL DEFAULT NULL,
  FOREIGN KEY (product_id) REFERENCES products(id) ON DELETE CASCADE,
  INDEX idx_product_images_product (product_id)
);

-- ── addresses ───────────────────────────────────────
CREATE TABLE addresses (
  id           BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  user_id      BIGINT UNSIGNED NOT NULL,
  first_name   VARCHAR(100) NOT NULL,
  last_name    VARCHAR(100) NOT NULL,
  phone        VARCHAR(30)  NOT NULL,
  address_line VARCHAR(300) NOT NULL,
  city         VARCHAR(100) NOT NULL,
  quartier     VARCHAR(100),
  zip_code     VARCHAR(20),
  country_code CHAR(3)      NOT NULL DEFAULT 'MAR',
  is_default   BOOLEAN      NOT NULL DEFAULT FALSE,
  created_at   TIMESTAMP    NULL DEFAULT NULL,
  updated_at   TIMESTAMP    NULL DEFAULT NULL,
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
  INDEX idx_addresses_user (user_id)
);

-- ── shipping_methods ────────────────────────────────
CREATE TABLE shipping_methods (
  id          BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  name        VARCHAR(100)  NOT NULL,
  description VARCHAR(200),
  price       DECIMAL(10,2) NOT NULL DEFAULT 0,
```

```
  zone        VARCHAR(100),
  is_active   BOOLEAN       NOT NULL DEFAULT TRUE,
  created_at  TIMESTAMP     NULL DEFAULT NULL,
  updated_at  TIMESTAMP     NULL DEFAULT NULL
);

-- ── coupons ──────────────────────────────────────────
CREATE TABLE coupons (
  id          BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  code        VARCHAR(50)   NOT NULL UNIQUE,
  type        ENUM('fixed','percent') NOT NULL,
  value       DECIMAL(10,2) NOT NULL,
  min_order   DECIMAL(10,2),
  usage_limit INT UNSIGNED,
  used_count  INT UNSIGNED  NOT NULL DEFAULT 0,
  expires_at  TIMESTAMP     NULL,
  is_active   BOOLEAN       NOT NULL DEFAULT TRUE,
  created_at  TIMESTAMP     NULL DEFAULT NULL,
  updated_at  TIMESTAMP     NULL DEFAULT NULL,
  INDEX idx_coupons_code (code)
);

-- ── orders ───────────────────────────────────────────
CREATE TABLE orders (
  id                 BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  order_number       VARCHAR(20)   NOT NULL UNIQUE,
  user_id            BIGINT UNSIGNED NULL,
  address_id         BIGINT UNSIGNED NOT NULL,
  shipping_method_id BIGINT UNSIGNED NOT NULL,
  coupon_id          BIGINT UNSIGNED NULL,
  status             ENUM('pending','confirmed','dispatched','shipped','delivered','cancelled')
  subtotal           DECIMAL(10,2) NOT NULL,
  shipping_cost      DECIMAL(10,2) NOT NULL DEFAULT 0,
  discount           DECIMAL(10,2) NOT NULL DEFAULT 0,
  total              DECIMAL(10,2) NOT NULL,
  notes              TEXT,
  created_at         TIMESTAMP     NULL DEFAULT NULL,
  updated_at         TIMESTAMP     NULL DEFAULT NULL,
  deleted_at         TIMESTAMP     NULL DEFAULT NULL,
  FOREIGN KEY (user_id)            REFERENCES users(id)            ON DELETE SET NULL,
  FOREIGN KEY (address_id)         REFERENCES addresses(id)        ON DELETE RESTRICT,
  FOREIGN KEY (shipping_method_id) REFERENCES shipping_methods(id) ON DELETE RESTRICT,
  FOREIGN KEY (coupon_id)          REFERENCES coupons(id)          ON DELETE SET NULL,
  INDEX idx_orders_user   (user_id),
  INDEX idx_orders_status (status),
  INDEX idx_orders_number (order_number)
);

-- ── order_items ──────────────────────────────────────
CREATE TABLE order_items (
  id           BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  order_id     BIGINT UNSIGNED NOT NULL,
  product_id   BIGINT UNSIGNED NOT NULL,
  product_name VARCHAR(200)   NOT NULL,
  unit_price   DECIMAL(10,2)  NOT NULL,
```

```
  quantity     SMALLINT UNSIGNED NOT NULL,
  created_at   TIMESTAMP      NULL DEFAULT NULL,
  updated_at   TIMESTAMP      NULL DEFAULT NULL,
  FOREIGN KEY (order_id)   REFERENCES orders(id)   ON DELETE CASCADE,
  FOREIGN KEY (product_id) REFERENCES products(id) ON DELETE RESTRICT,
  INDEX idx_order_items_order (order_id)
);

-- ── order_status_histories ──────────────────────────
CREATE TABLE order_status_histories (
  id         BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  order_id   BIGINT UNSIGNED NOT NULL,
  status     ENUM('pending','confirmed','dispatched','shipped','delivered','cancelled') NOT NULL
  note       VARCHAR(300),
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (order_id) REFERENCES orders(id) ON DELETE CASCADE,
  INDEX idx_osh_order (order_id)
);

-- ── wishlist_items ──────────────────────────────────
CREATE TABLE wishlist_items (
  id         BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  user_id    BIGINT UNSIGNED NOT NULL,
  product_id BIGINT UNSIGNED NOT NULL,
  created_at TIMESTAMP       NOT NULL DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (user_id)    REFERENCES users(id)    ON DELETE CASCADE,
  FOREIGN KEY (product_id) REFERENCES products(id) ON DELETE CASCADE,
  UNIQUE KEY uq_wishlist (user_id, product_id)
);

-- ── reviews ─────────────────────────────────────────
CREATE TABLE reviews (
  id            BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  user_id       BIGINT UNSIGNED NOT NULL,
  product_id    BIGINT UNSIGNED NOT NULL,
  order_item_id BIGINT UNSIGNED NOT NULL,
  rating        TINYINT UNSIGNED NOT NULL CHECK (rating BETWEEN 1 AND 5),
  body          TEXT,
  is_approved   BOOLEAN NOT NULL DEFAULT FALSE,
  created_at    TIMESTAMP NULL DEFAULT NULL,
  updated_at    TIMESTAMP NULL DEFAULT NULL,
  FOREIGN KEY (user_id)       REFERENCES users(id)       ON DELETE CASCADE,
  FOREIGN KEY (product_id)    REFERENCES products(id)    ON DELETE CASCADE,
  FOREIGN KEY (order_item_id) REFERENCES order_items(id) ON DELETE CASCADE,
  UNIQUE KEY uq_review (user_id, order_item_id),
  INDEX idx_reviews_product (product_id),
  INDEX idx_reviews_approved (is_approved)
);
```

6. Backend Architecture — Laravel

6.1 Directory Structure

```
app/
├── Http/
│   ├── Controllers/
│   │   └── Api/
│   │       ├── Auth/
│   │       │   └── AuthController.php
│   │       ├── ProductController.php
│   │       ├── BrandController.php
│   │       ├── CategoryController.php
│   │       ├── CartController.php
│   │       ├── WishlistController.php
│   │       ├── OrderController.php
│   │       ├── CheckoutController.php
│   │       ├── ReviewController.php
│   │       ├── CouponController.php
│   │       ├── ShippingMethodController.php
│   │       └── Admin/
│   │           ├── ProductController.php
│   │           ├── OrderController.php
│   │           └── UserController.php
│   │
│   ├── Requests/
│   │   ├── Auth/LoginRequest.php
│   │   ├── Auth/RegisterRequest.php
│   │   ├── CheckoutRequest.php
│   │   ├── ReviewRequest.php
│   │   └── Admin/StoreProductRequest.php
│   │
│   ├── Resources/
│   │   ├── UserResource.php
│   │   ├── ProductResource.php
│   │   ├── ProductCollectionResource.php
│   │   ├── OrderResource.php
│   │   ├── OrderDetailResource.php
│   │   ├── ReviewResource.php
│   │   └── WishlistItemResource.php
│   │
│   └── Middleware/
│       ├── ForceJsonResponse.php
│       └── EnsureRole.php
│
├── Models/
│   ├── User.php
│   ├── Product.php
│   ├── Brand.php
│   ├── Category.php
│   ├── Address.php
│   ├── Order.php
│   ├── OrderItem.php
```

```
│   ├── OrderStatusHistory.php
│   ├── WishlistItem.php
│   ├── Review.php
│   ├── ShippingMethod.php
│   └── Coupon.php
│
├── Services/
│   ├── CartService.php
│   ├── OrderService.php
│   ├── CouponService.php
│   └── ReviewService.php
│
├── Policies/
│   ├── OrderPolicy.php
│   ├── ReviewPolicy.php
│   └── WishlistPolicy.php
│
└── Jobs/
    └── SendOrderConfirmationEmail.php
```

6.2 Layer Responsibilities

Controllers — Thin. Receive HTTP, delegate to Services, return Resource responses. No business logic.

Services — All business logic lives here.

- `CartService` — Handles cart session/DB reconciliation, coupon application, total calculation.
- `OrderService` — Converts cart to order, decrements stock, triggers email job, creates status history entry.
- `CouponService` — Validates coupon applicability, calculates discount.
- `ReviewService` — Validates user has purchased product before allowing review.

Requests (FormRequest) — Server-side validation with typed rules. Every mutation goes through a FormRequest. Reuses `$messages` and `$attributes` for French/Arabic error messages.

Resources (API Resources) — Shape the JSON output. Prevents over-exposure of sensitive fields (`password`, internal flags). Use `ProductCollectionResource` (lean) vs `ProductDetailResource` (full) for list vs detail.

Policies — Authorization logic isolated from controllers:

- `OrderPolicy` — `view()` ensures user owns the order.
- `ReviewPolicy` — `create()` checks the order item belongs to the authenticated user and has no existing review.

Middleware

- `ForceJsonResponse` — Sets `Accept: application/json` globally to prevent HTML error pages bleeding into the API.
- `EnsureRole` — Protects admin routes: `middleware('role:admin')`.

Jobs

- `SendOrderConfirmationEmail` — Dispatched to queue after order creation; prevents checkout delay.

---

7. API Design Contract

> *Base URL:* `https://api.bloomparfums.ma/api`
> *All responses follow* `{ data, message, meta? }` *envelope.*
> *Auth header:* `Authorization: Bearer <token>`

Authentication

```
POST /auth/register
POST /auth/login
POST /auth/logout          [auth]
POST /auth/refresh         [auth]
GET  /auth/me              [auth]
```

POST /auth/login

```
Request:  { "email": "user@example.com", "password": "secret" }
Response: {
  "data": {
    "token": "eyJ...",
    "token_type": "Bearer",
    "expires_in": 86400,
    "user": { "id": 1, "name": "Ayoub", "email": "...", "role": "customer" }
  }
}
Errors: 422 (validation), 401 (invalid credentials)
```

Products

```
GET  /products                    → Paginated list with filters
GET  /products/{slug}             → Single product detail
GET  /products/featured           → BestSellers section
```

GET /products

```
Query params:
  search=string
  brand_id=int[]
  category_id=int[]
  gender=men|women|unisex|kids
  price_min=decimal
  price_max=decimal
  sort=price_asc|price_desc|newest|rating
  page=int
  per_page=int (default 20, max 50)

Response headers: X-Total-Count, Link (pagination)
```

Brands & Categories

```
GET  /brands       → List all active brands (for filter dropdown)
GET  /categories   → Tree of categories
```

Wishlist `[auth]`

```
GET    /wishlist              → List user's wishlist items
POST   /wishlist              → { product_id: int }
DELETE /wishlist/{product_id} → Remove item
```

Cart `[auth or session]`

```
GET    /cart          → Current cart contents + totals
POST   /cart/items    → { product_id, quantity }
PATCH  /cart/items/{id} → { quantity }
DELETE /cart/items/{id}
POST   /cart/coupon   → { code: "PROMO10" }
DELETE /cart/coupon
```

GET /cart response:

```
{
  "data": {
    "items": [
      {
        "id": 1,
        "product": { "id": 1, "name": "SUGAR POP", "price": 140, "image_url": "…" },
        "quantity": 1,
        "line_total": 140
      }
    ],
    "subtotal": 240,
    "shipping_cost": 35,
    "discount": 0,
    "total": 275,
    "coupon": null
  }
}
```

Shipping Methods

```
GET /shipping-methods    → List active methods with prices
```

Checkout & Orders `[auth]`

```
GET  /addresses           → User saved addresses
POST /addresses           → Create/save new address
PUT  /addresses/{id}

POST /checkout            → Create order from cart
GET  /orders              → User order history (paginated)
GET  /orders/{orderNumber} → Order detail + items + status history
```

POST /checkout

```
Request:
{
  "address_id": 3,
  "shipping_method_id": 2,
  "coupon_code": "PROMO10"
}

Response 201:
{
  "data": {
    "order_number": "LX-8921-Q",
    "status": "confirmed",
    "total": 275,
    "items": [ ... ],
    "estimated_delivery": "2026-02-28"
  }
}

Errors:
  422 — Validation failure
  409 — Stock insufficient for one or more items
  410 — Coupon expired or exhausted
```

GET /orders/{orderNumber}

```
{
  "data": {
    "order_number": "LX-8921-Q",
    "status": "delivered",
    "status_history": [
      { "status": "confirmed",  "created_at": "2026-02-22T10:30:00Z" },
      { "status": "dispatched", "created_at": "2026-02-23T12:30:00Z" },
      { "status": "shipped",    "created_at": "2026-02-23T17:30:00Z" },
      { "status": "delivered",  "created_at": "2026-02-24T09:00:00Z" }
    ],
    "items": [ ... ],
    "shipping_address": { ... },
    "total": 275
  }
}
```

Reviews `[auth]`

```
POST /reviews              → Submit review for an order item
GET  /products/{slug}/reviews → Public list of approved reviews
```

POST /reviews

```
Request (multipart/form-data):
{
  "order_item_id": 12,
  "rating": 5,
  "body": "Excellent fragrance, lasts 8+ hours.",
  "images[]": [File]
}

Errors:
  403 – User did not purchase this product
  409 – Review already submitted for this order item
  422 – Validation failure
```

Admin Endpoints `[auth + role:admin]`

```
GET    /admin/products
POST   /admin/products
PUT    /admin/products/{id}
DELETE /admin/products/{id}

GET    /admin/orders
PATCH  /admin/orders/{id}/status   → { status: "shipped", note: "Shipped via Amana" }

GET    /admin/users
GET    /admin/coupons
POST   /admin/coupons
PATCH  /admin/coupons/{id}
```

---

8. Security & Scalability

8.1 Authentication Strategy

Use **Laravel Sanctum** with token-based auth (not cookie-based SPA mode) since the frontend and backend are on separate domains. Token TTL should be 24 hours, with a refresh endpoint as already implemented in `api.ts`.

Do **not** store the JWT in `localStorage`. The current implementation using `js-cookie` with `secure: true, sameSite: strict` is the correct approach. Consider moving to `HttpOnly` cookies via a Next.js Route Handler proxy to eliminate XSS exposure entirely.

8.2 Rate Limiting

Apply differentiated rate limits per route group:

```
// routes/api.php
Route::middleware('throttle:auth')->group(fn() => [
  // login: 5 attempts/minute per IP
]);

Route::middleware('throttle:api')->group(fn() => [
  // general endpoints: 120 requests/minute per user
]);
```

Implement exponential back-off on login failures at the application level to deter brute force.

8.3 Input Validation Risks

- **Checkout:** Validate that `address_id` belongs to the authenticated user before creating the order. Without this, IDOR allows ordering to arbitrary addresses.
- **Review:** Enforce `order_item_id` belongs to the current user at policy level. The `UNIQUE(user_id, order_item_id)` DB constraint is the last line of defense.
- **Coupon:** Validate atomically — check + increment `used_count` inside a DB transaction to prevent race conditions.
- **Price tampering:** Never accept prices from the frontend. Prices are always resolved from the database on the server side.

8.4 N+1 Query Risks

Risk location	Mitigation
Product listing with brand + category	`Product::with(['brand', 'category'])`
Order detail with items + product	`Order::with(['items.product', 'statusHistory'])`
Wishlist with product images	`WishlistItem::with('product.images')`
Review listing with user	`Review::with('user:id,name,avatar_url')`

Use Laravel API Resources with `whenLoaded()` to guarantee relationships are never lazy-loaded inside a resource.

8.5 Caching Opportunities

Data	Strategy	TTL
Featured products	Redis `Cache::remember`	1 hour
Brand list	Redis `Cache::remember`	6 hours
Category tree	Redis `Cache::remember`	6 hours
Product detail	Cache per slug, invalidate on update	30 min
Shipping methods	Cache	1 day

Use **cache tags** to invalidate product cache on admin update without flushing all keys.

8.6 File Upload Handling (Review Photos)

- Accept only `jpeg, png, webp` with max 5 MB per file.
- Validate MIME type from file content, not extension.
- Store on **AWS S3** or **Cloudflare R2** via Laravel's `Storage::disk('s3')`.
- Process image resizing via a queued job to avoid blocking the API response.
- Serve via a CDN URL, not direct S3 links.

8.7 CORS Configuration

```
// config/cors.php
'allowed_origins' => [env('FRONTEND_URL', 'https://bloomparfums.ma')],
'allowed_methods' => ['GET', 'POST', 'PUT', 'PATCH', 'DELETE', 'OPTIONS'],
'allowed_headers' => ['Content-Type', 'Authorization', 'Accept'],
'supports_credentials' => true,
```

8.8 Environment Separation

ENV	Frontend URL	API URL	Debug	Cache
local	localhost:3000	localhost:8000	true	file
staging	staging.bloomparfums.ma	api-staging.bloomparfums.ma	false	Redis
production	bloomparfums.ma	api.bloomparfums.ma	false	Redis + CDN

9. Final Recommendations

Priority 1 — Unblock Commerce (Week 1-2)

1. **Build `/product/[id]` page.** This is the core conversion page and is entirely missing. Priority zero.
2. **Implement real Cart state.** Replace hardcoded `CartDrawer` with a React Context or Zustand store, synced to a Laravel cart API (or `localStorage` as a fast interim with server merge on login).
3. **Wire Checkout to backend.** `POST /checkout` endpoint and connect the form with real validation.
4. **Build Order confirmation with real data.** Connect `/success` to the order response from checkout.

Priority 2 — Complete the Post-Purchase Chain (Week 2-3)

5. **Real order status from API.** `/order-status` page should fetch from `GET /orders/{orderNumber}` and render the real `status_history` array in the timeline.
6. **Wire review submission.** `ReviewModal` should `POST /reviews` with multipart form data.
7. **Wishlist persistence.** Call `POST /wishlist` on heart icon click; `GET /wishlist` on page load.

Priority 3 — Foundation Quality (Week 3-4)

8. **Add `error.tsx` and `loading.tsx`** at app-segment level for every route.
9. **Migrate filter state to URL params.** Use `useSearchParams` so filtered collection URLs are shareable and bookmarkable.
10. **Fix histogram `Math.random()` in Collection page.** Replace with static seed data or a stable distribution algorithm.
11. **Implement focus traps in all modals** (CartDrawer, FilterModal, ReviewModal).
12. **Add `sizes` prop to all next/Image components** in product grids.

Priority 4 — Admin Panel (Month 2)

13. Build a `/admin` route group with its own layout behind `EnsureRole` middleware.
14. Implement product CRUD, order status management, and review moderation.
15. Integrate a basic analytics dashboard (orders per day, revenue, top products).

Architecture Decision Record

Decision	Recommendation	Rationale
Auth storage	`HttpOnly` cookie via Next.js API Route proxy	Eliminates XSS token theft risk
Cart persistence	Backend DB cart (not localStorage)	Required for cross-device sync and abandoned cart recovery
State management	Zustand (lightweight)	React Context causes full-tree rerenders on cart updates
Image CDN	Cloudflare R2 + CDN	Cost-effective, globally distributed, integrates with Next.js `remotePatterns`
Real-time order status	Polling every 30s via SWR `refreshInterval`	Sufficient for e-commerce; WebSocket is over-engineering at this stage
Review media	Queued job → S3	Keeps API latency under 200ms regardless of image size

---

Report generated by AI Lead Architect | Bloom Parfums E-commerce Platform | Feb 2026
